CrewAI Implementation Report

Smart Supply Chain Optimizer — Multi-Agent Crisis Resolution System

Student: Deniz Büyükşahin | **Git:** github.com/Dnzbykshn/SmartSupplyChainOptimizer

Technology Stack: CrewAI + Google Gemini + FastAPI + Supabase + Next.js

---

1. Project Overview & How CrewAI Adds Functionality

This project extends the foundational Next.js website (Homework 2) by integrating a **full CrewAI Multi-Agent AI backend** that transforms the static UI into a live, AI-powered crisis resolution platform.

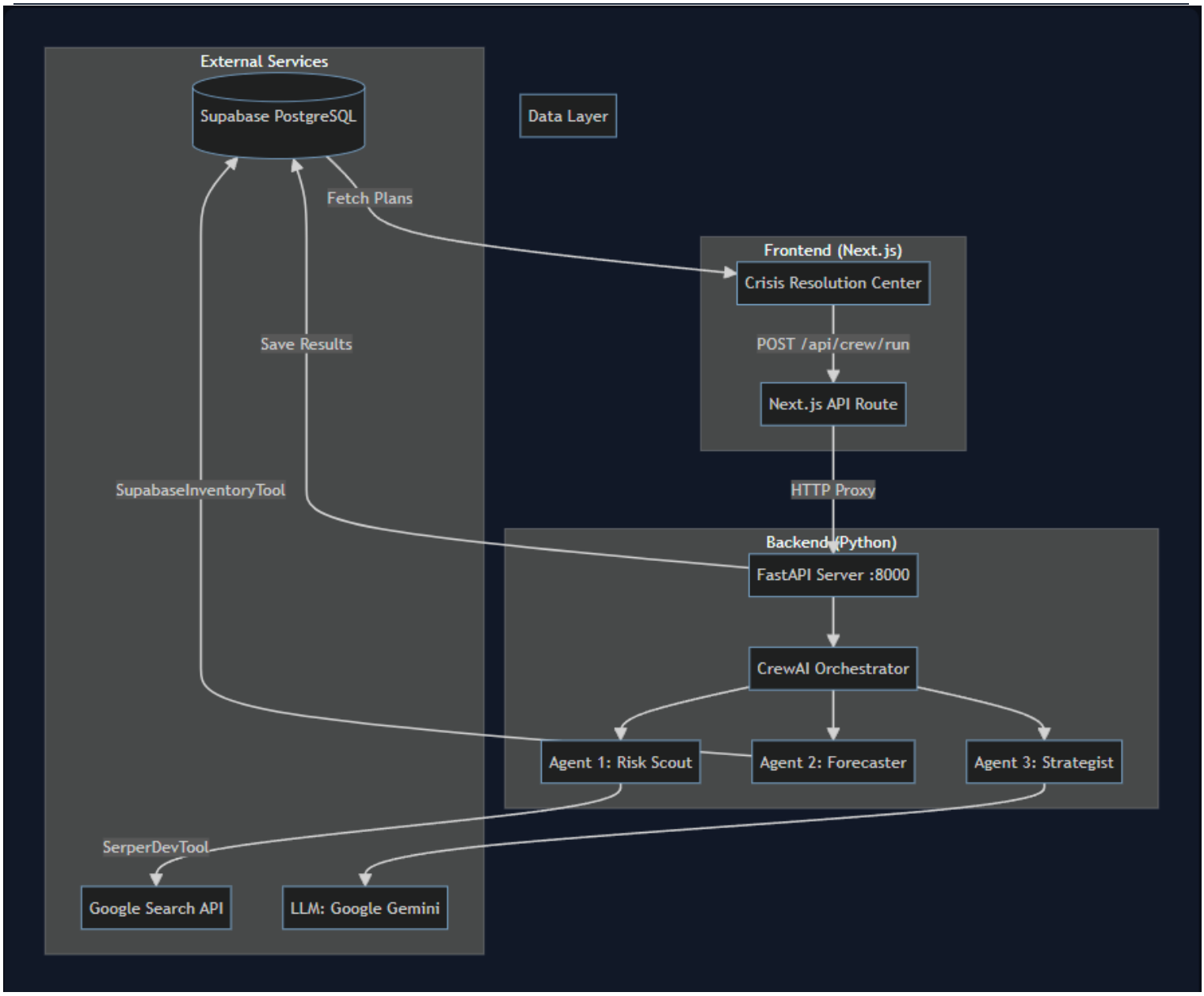
What Homework 2 Provided (Static Frontend)

- A dashboard with mocked shipment data and placeholder alerts
- A Crisis Resolution Center with hardcoded risk data and mock mitigation plans
- A Settings page with client-side state (no persistence)

What CrewAI Adds (Live AI Backend)

- **3 specialized AI agents** that autonomously analyze real-world crises using live web search
- **Real-time inventory querying** from a PostgreSQL database (Supabase)
- **Structured JSON output** (Pydantic models) that maps directly to frontend TypeScript interfaces
- **Database persistence** — every crisis run is saved with full audit trail
- **User settings integration** — corporate parameters dynamically influence AI decisions
- **FastAPI REST API** — bridges the Next.js frontend with the Python AI orchestration layer

System Architecture



2. Agent Definitions

2.1 Configuration File: agents.yaml

The agents are defined in a YAML configuration file, following CrewAI's `@CrewBase` decorator pattern. Each agent has a specialized **role**, **goal**, and **backstory**.

```
# agents.yaml — Multi-Agent AI Definitions
#
# Three specialized supply chain agents, each with a distinct role,
# goal, and backstory. Placeholders ({crisis_description}, etc.) are
# injected at runtime via crew.kickoff(inputs={...}).

global_risk_scout:
  role: >
    Global Supply Chain Risk Intelligence Analyst
  goal: >
    Analyze the following crisis event and produce a comprehensive threat
    assessment covering geopolitical factors, weather risks, affected logistics
    infrastructure, and estimated disruption duration:
    {crisis_description}
  backstory: >
    You are a former intelligence analyst who transitioned into global supply
    chain risk management. With 15 years of experience monitoring geopolitical
    disruptions, maritime chokepoints, and labor disputes across Asia-Pacific
    trade corridors, you have an unmatched ability to synthesize breaking news,
    weather data, and port authority reports into actionable threat assessments.
    You were the first analyst to predict the 2024 Red Sea shipping crisis
    three weeks before major carriers began rerouting. Your reports are trusted
    by Fortune 500 logistics directors worldwide.

inventory_forecaster:
  role: >
    Inventory Depletion & Production Impact Forecaster
  goal: >
    Based on the risk assessment from the Global Risk Scout, project the
    inventory depletion timeline for critical components and quantify the
    revenue and production line impact of the current crisis.
    Current inventory status: {current_inventory_status}
  backstory: >
    You are a senior operations research scientist specializing in
    stochastic inventory modeling and just-in-time supply chain analytics.
    You spent 10 years at a global electronics manufacturer optimizing
    safety stock levels across 12 regional warehouses. You are known for
    your precise daily burn-rate calculations and your ability to project
    exactly when production lines will halt due to component shortages.
    Your forecasting models have saved companies over $200M in avoided
    stockout losses.

routing_strategist:
  role: >
    Logistics Routing & Mitigation Plan Strategist
  goal: >
    Using the risk assessment and inventory impact analysis, generate exactly
    3 actionable mitigation plans with varying cost/risk/speed profiles.
    Each plan must include precise cost estimates, timeframes, reliability
    scores, and CO2 impact assessments.
    Affected routes: {affected_routes}
```

```
  backstory: >
    You are a chief logistics strategist who has orchestrated emergency
    supply chain rerouting for multinational corporations during every major
    disruption of the past decade. You have deep knowledge of alternative
    shipping corridors including the China-Europe railway network,
    transshipment hubs in Southeast Asia, and charter air freight capacity.
```

2.2 Agent Descriptions

Agent	Role	Tool	Purpose
Global Risk Scout	Risk Intelligence Analyst	`SerperDevTool` (Web Search)	Scans real-time news to produce threat assessments
Inventory Forecaster	Depletion & Impact Analyst	`SupabaseInventoryTool` (Database)	Queries live inventory to calculate burn rates and revenue at risk
Routing Strategist	Mitigation Plan Advisor	*(None — uses context)*	Combines risk + inventory data to generate 3 costed mitigation plans

---

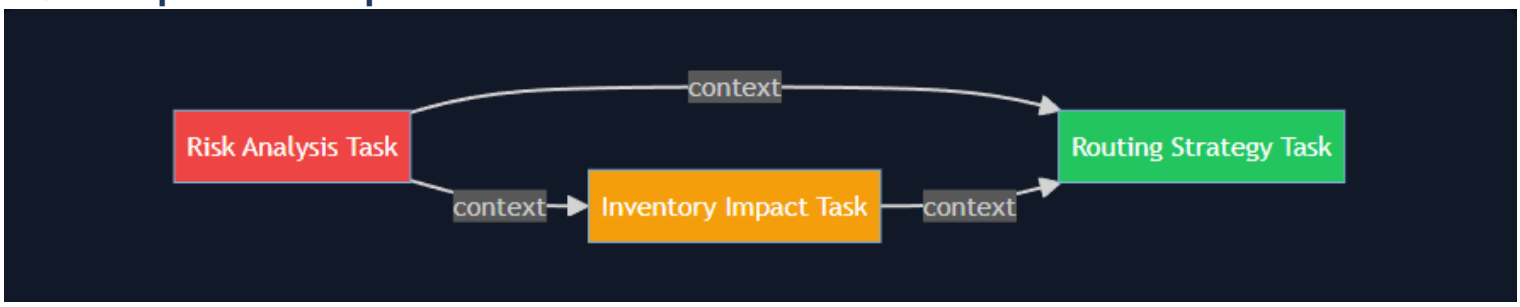
3. Task Definitions

3.1 Configuration File: tasks.yaml

```
# tasks.yaml — Sequential Task Pipeline Definitions
#
# Three tasks forming a sequential chain:
#   risk_analysis_task → inventory_impact_task → routing_strategy_task

risk_analysis_task:
  description: >
    Conduct a comprehensive risk analysis of the following global supply
    chain disruption:
    CRISIS EVENT: {crisis_description}
    Your analysis must cover:
    1. Threat severity classification (Critical / High / Medium / Low)
    2. Affected ports and logistics infrastructure with IATA/UNLOC codes
    3. Current weather conditions and secondary environmental risks
    4. Relevant news intelligence from major financial outlets
    5. Geopolitical factors that may accelerate or delay resolution
    6. Estimated disruption duration with conservative and optimistic scenarios
  expected_output: >
    A structured JSON object with keys: threatLevel, eventName,
    eventDescription, affectedPorts, weatherConditions, newsHeadlines,
    geopoliticalFactors, estimatedDuration
  agent: global_risk_scout

inventory_impact_task:
  description: >
    Based on the Global Risk Scout's threat assessment, analyze the
    inventory impact of this supply chain disruption.
```

```
    CURRENT INVENTORY STATUS: {current_inventory_status}
    {user_settings}
    IMPORTANT: Use the "Minimum Safety Stock" parameter from the corporate
    configuration. Any component with fewer days of supply remaining than
    the configured safety stock threshold should be flagged as CRITICAL.
  expected_output: >
    A structured JSON object with keys: criticalItems, totalSkusAffected,
    revenueAtRisk, productionLineImpact
  agent: inventory_forecaster

routing_strategy_task:
  description: >
    Using both the risk assessment and inventory impact analysis, generate
    exactly 3 mitigation plans for the supply chain crisis.
    AFFECTED ROUTES: {affected_routes}
    {user_settings}
    Plans MUST follow this pattern:
    - Plan A: Conservative / lowest cost / highest risk
    - Plan B: Balanced / moderate cost / moderate risk
    - Plan C: Aggressive / highest cost / lowest risk
  expected_output: >
    A JSON object with a "plans" array containing exactly 3 mitigation
    plan objects with all 10 required fields.
  agent: routing_strategist
```

3.2 Sequential Pipeline Flow



Each task's output is passed as **context** to subsequent tasks, creating a data pipeline where:

1. The **Risk Scout** searches the internet and produces a threat assessment
2. The **Forecaster** receives the threat data AND queries the live database
3. The **Strategist** receives BOTH previous outputs to formulate plans

---

4. Crew Orchestration Code (crew.py)

This is the central orchestration file that wires YAML configs to Python agent/task definitions using CrewAI's `@CrewBase` decorator pattern:

```
from crewai import Agent, Crew, Process, Task, LLM
from crewai.project import CrewBase, agent, crew, task
from crewai_tools import SerperDevTool
from supply_chain_crew.models import MitigationPlanList, RiskScoutOutput,
InventoryImpactOutput
from supply_chain_crew.tools import SupabaseInventoryTool

gemini_llm = LLM(model="gemini/gemini-3-flash-preview")
```

```
search_tool = SerperDevTool()
inventory_tool = SupabaseInventoryTool()

@CrewBase
class SupplyChainCrew:
    """Smart Supply Chain Optimizer — Multi-Agent Crisis Resolution Crew"""
    agents_config = "config/agents.yaml"
    tasks_config = "config/tasks.yaml"

    @agent
    def global_risk_scout(self) -> Agent:
        return Agent(
            config=self.agents_config["global_risk_scout"],
            verbose=True, allow_delegation=False,
            llm=gemini_llm, tools=[search_tool],
        )

    @agent
    def inventory_forecaster(self) -> Agent:
        return Agent(
            config=self.agents_config["inventory_forecaster"],
            verbose=True, allow_delegation=False,
            llm=gemini_llm, tools=[inventory_tool],
        )

    @agent
    def routing_strategist(self) -> Agent:
        return Agent(
            config=self.agents_config["routing_strategist"],
            verbose=True, allow_delegation=False,
            llm=gemini_llm,
        )

    @task
    def risk_analysis_task(self) -> Task:
        return Task(config=self.tasks_config["risk_analysis_task"],
                    output_pydantic=RiskScoutOutput)

    @task
    def inventory_impact_task(self) -> Task:
        return Task(config=self.tasks_config["inventory_impact_task"],
                    context=[self.risk_analysis_task()],
                    output_pydantic=InventoryImpactOutput)

    @task
    def routing_strategy_task(self) -> Task:
        return Task(config=self.tasks_config["routing_strategy_task"],
                    context=[self.risk_analysis_task(), self.inventory_impact_task()],
                    output_pydantic=MitigationPlanList)

    @crew
    def crew(self) -> Crew:
        return Crew(agents=self.agents, tasks=self.tasks,
                    process=Process.sequential, verbose=True)
```

Key Design Decisions:

- `Process.sequential`: Agents run in order (Scout → Forecaster → Strategist), not in parallel
- `allow_delegation=False`: Each agent completes its own task; no cross-delegation
- `output_pydantic`: Each task produces validated, typed JSON output
- `context` **chaining**: Task 2 receives Task 1's output; Task 3 receives both

---

5. Kickoff Code (main.py)

```
from supply_chain_crew.crew import SupplyChainCrew

def main():
    print("=" * 70)
    print("  SMART SUPPLY CHAIN OPTIMIZER — Multi-Agent Crisis Resolution")
    print("=" * 70)

    supabase = get_supabase_client()
    user_settings = fetch_user_settings(supabase)
    CRISIS_INPUTS["user_settings"]  =  format_settings_for_prompt(user_settings)

    crew_instance = SupplyChainCrew().crew()
    result = crew_instance.kickoff(inputs=CRISIS_INPUTS)

    plans_data = None
    if result.pydantic:
        plans_data = result.pydantic.model_dump()
        output_json = json.dumps(plans_data, indent=2, ensure_ascii=False)
        with open("output_plan.json", "w", encoding="utf-8") as f:
            f.write(output_json)

    agent_outputs = {}
    if hasattr(result, 'tasks_output') and result.tasks_output:
        tasks = result.tasks_output
        if len(tasks) > 0:
            if tasks[0].pydantic:
                agent_outputs["risk_scout"] = json.dumps(
                    tasks[0].pydantic.model_dump(),  ensure_ascii=False)
            else:
                agent_outputs["risk_scout"] = tasks[0].raw
        if len(tasks) > 1:
            if tasks[1].pydantic:
                agent_outputs["forecaster"] = json.dumps(
                    tasks[1].pydantic.model_dump(),  ensure_ascii=False)
            else:
                agent_outputs["forecaster"]  =  tasks[1].raw

    if plans_data and supabase:
        save_to_supabase(supabase,  plans_data,  CRISIS_INPUTS,  agent_outputs)

if __name___ == "__main__":
    main()
```

Dynamic Inputs (Runtime Variables)

```
CRISIS_INPUTS = {
    "crisis_description": "Search the internet for the most recent news
        regarding shipping delays and disruptions in the Red Sea and
        Suez Canal area...",
    "affected_routes": "Shenzhen → Rotterdam (via Red Sea),
        Mumbai → Genoa (via Suez Canal)",
    "current_inventory_status": "Query the Supabase Database using your
        Query Supabase Inventory tool...",
    "user_settings": "(loaded from database at runtime)",
}
```

---

6. Pydantic Structured Output Models (models.py)

All three agents produce **type-safe JSON** via Pydantic models:

```
from typing import List, Literal
from pydantic import BaseModel, Field

class RiskScoutOutput(BaseModel):
    threatLevel: Literal["Critical", "High", "Medium", "Low"]
    eventName: str
    eventDescription: str
    affectedPorts: List[str]
    weatherConditions: str
    newsHeadlines: List[str]
    geopoliticalFactors: List[str]
    estimatedDuration: str

class CriticalItem(BaseModel):
    name: str
    currentStock: int
    dailyBurnRate: int
    daysUntilDepletion: int
    reorderPoint: int
    status: Literal["Critical", "Warning", "Stable"]

class InventoryImpactOutput(BaseModel):
    criticalItems: List[CriticalItem]
    totalSkusAffected: int
    revenueAtRisk: str
    productionLineImpact: str

class MitigationPlan(BaseModel):
    id: str
    name: str
    strategy: str
    description: str
    estimatedCost: str
    timeframe: str
    riskLevel: Literal["Low", "Medium", "High"]
    co2Impact: str
    reliabilityScore: int = Field(ge=0, le=100)
```

```
    details: List[str]

class MitigationPlanList(BaseModel):
    plans: List[MitigationPlan]
```

---

7. Custom Tool: Supabase Inventory Query (tools.py)

```
import os
from crewai.tools import BaseTool
from supabase import create_client

class SupabaseInventoryTool(BaseTool):
    name: str = "Query Supabase Inventory"
    description: str = (
        "Queries the Supabase 'inventory' table to get real-time "
        "stock levels, daily burn rates, and warehouse locations."
    )

    def _run(self) -> str:
        supabase_url = os.getenv("SUPABASE_URL")
        supabase_key = os.getenv("SUPABASE_SERVICE_ROLE_KEY")
        supabase = create_client(supabase_url, supabase_key)
        response = supabase.table("inventory").select("*").execute()

        result_str = "CURRENT INVENTORY DATA:\n"
        for item in response.data:
            days_left = item['current_stock'] / item['daily_burn_rate']
            result_str += f"- Item: {item['item_name']}\n"
            result_str += f"  Stock: {item['current_stock']} {item['unit']}\n"
            result_str += f"  Burn Rate: {item['daily_burn_rate']}/day\n"
            result_str += f"  Days Left: {days_left:.1f}\n"
            result_str += f"  Warehouse: {item['warehouse_location']}\n\n"
        return result_str
```

---

8. FastAPI Integration (api.py)

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from supply_chain_crew.crew import SupplyChainCrew
from supply_chain_crew.main import (
    get_supabase_client, save_to_supabase,
    fetch_user_settings, format_settings_for_prompt
)

app = FastAPI(title="Smart Supply Chain Optimizer API")

class CrisisPayload(BaseModel):
    crisis_description:  str
```

```
    affected_routes:  str

@app.post("/api/analyze-crisis")
async def analyze_crisis(payload: CrisisPayload):
    supabase = get_supabase_client()
    user_settings = fetch_user_settings(supabase)
    settings_context = format_settings_for_prompt(user_settings)

    crisis_inputs = {
        "crisis_description": payload.crisis_description,
        "affected_routes": payload.affected_routes,
        "current_inventory_status": "Query the Supabase Database...",
        "user_settings": settings_context,
    }

    crew_instance = SupplyChainCrew().crew()
    result = crew_instance.kickoff(inputs=crisis_inputs)
    plans_data = result.pydantic.model_dump() if result.pydantic else result.json_dict
    run_id = save_to_supabase(supabase, plans_data, crisis_inputs, agent_outputs)

    return {
        "status": "success",
        "run_id": run_id,
        "plans": plans_data.get("plans", []),
        "agent_outputs": agent_outputs,
    }
```

---

9. Sample Output (output_plan.json)

Below is an actual output generated by the CrewAI agents analyzing a Red Sea Maritime Disruption:

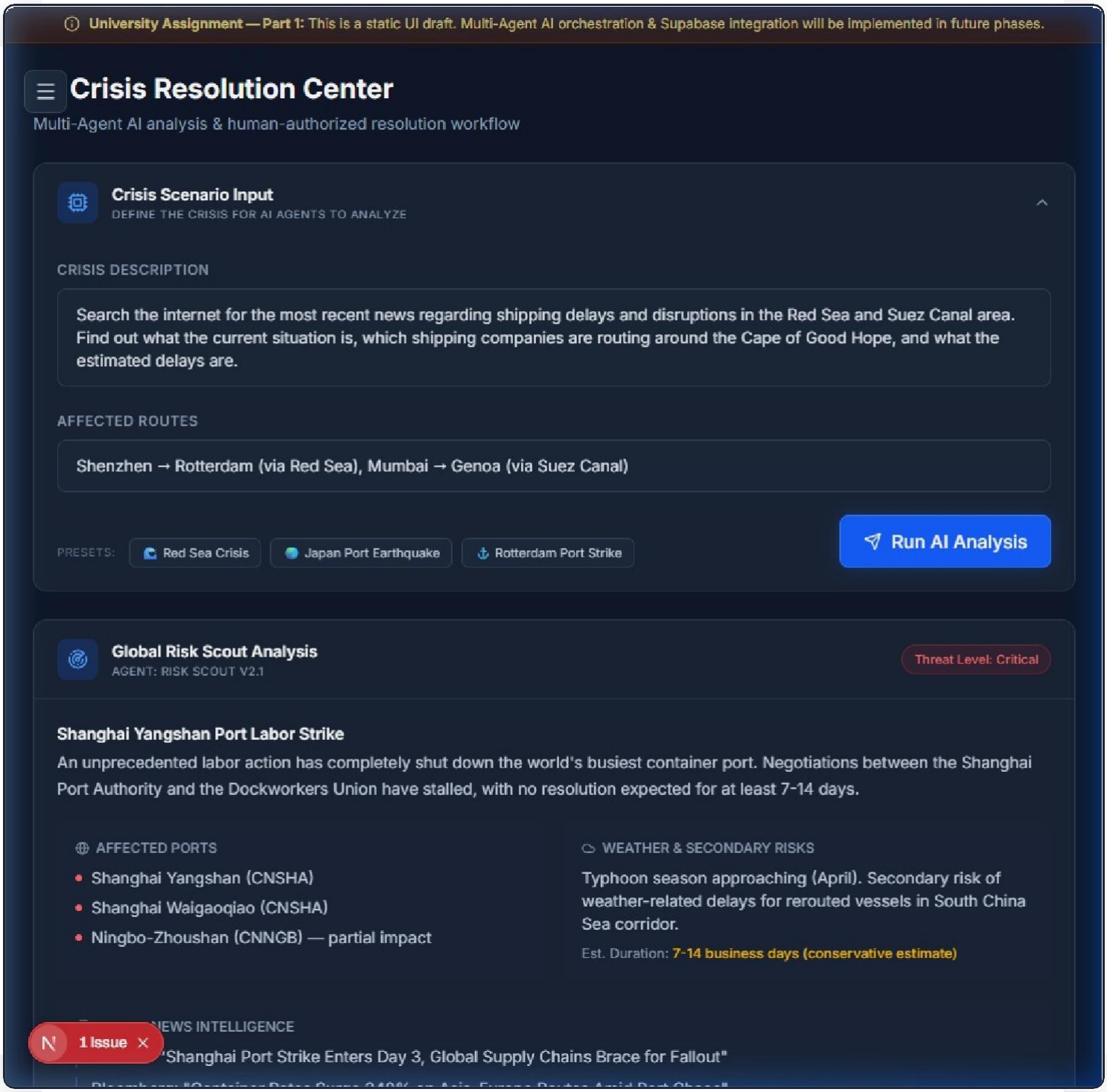

```
{
  "plans": [
    {
      "id": "plan-a",
      "name": "Plan A",
      "strategy": "Cape of Good Hope Diversion",
      "description": "Maintain the standard maritime rerouting around Africa.",
      "estimatedCost": "$28,000 — $35,000 per FEU",
      "timeframe": "45-50 days",
      "riskLevel": "High",
      "co2Impact": "+35% vs. original sea route",
      "reliabilityScore": 65,
      "details": [
        "Accepts a total production halt on Line Alpha.",
        "Avoids Suez-related War Risk Surcharges.",
        "Subject to extreme maritime weather risks.",
        "Relies on existing carrier capacity."
      ]
    },
    {
      "id": "plan-b",
      "name": "Plan B",
```

```
      "strategy": "China-Europe Rail & Sea-Air Intermodal",
      "description": "Utilize the China-Europe Railway Express and Sea-Air transshipment via Dubai.",
      "estimatedCost": "$145,000 — $195,000 per shipment",
      "timeframe": "18-24 days",
      "riskLevel": "Medium",
      "co2Impact": "-12% vs. COGH sea route",
      "reliabilityScore": 80,
      "details": [
        "Bypasses the Red Sea via rail corridors for OLED modules.",
        "Utilizes Jebel Ali (UAE) as a transshipment hub.",
        "Reduces projected revenue at risk by ~60%.",
        "Requires intensive customs coordination."
      ]
    },
    {
      "id": "plan-c",
      "name": "Plan C",
      "strategy": "Emergency Strategic Air-Bridge",
      "description": "Deploy dedicated charter flights to airlift critical components.",
      "estimatedCost": "$950,000 — $1,250,000 per flight",
      "timeframe": "3-5 days",
      "riskLevel": "Low",
      "co2Impact": "+850% vs. original sea route",
      "reliabilityScore": 98,
      "details": [
        "Immediate dispatch of 25,000 OLED modules.",
        "Provides 100% protection for the $26.24M revenue-at-risk.",
        "Bypasses all maritime chokepoints.",
        "Utilizes premium air-charter partners (DHL/DSV)."
      ]
    }
  ]
}
```


---

10. Project Screenshots & Explanations

10.1 Crisis Resolution Center — Scenario Input Panel

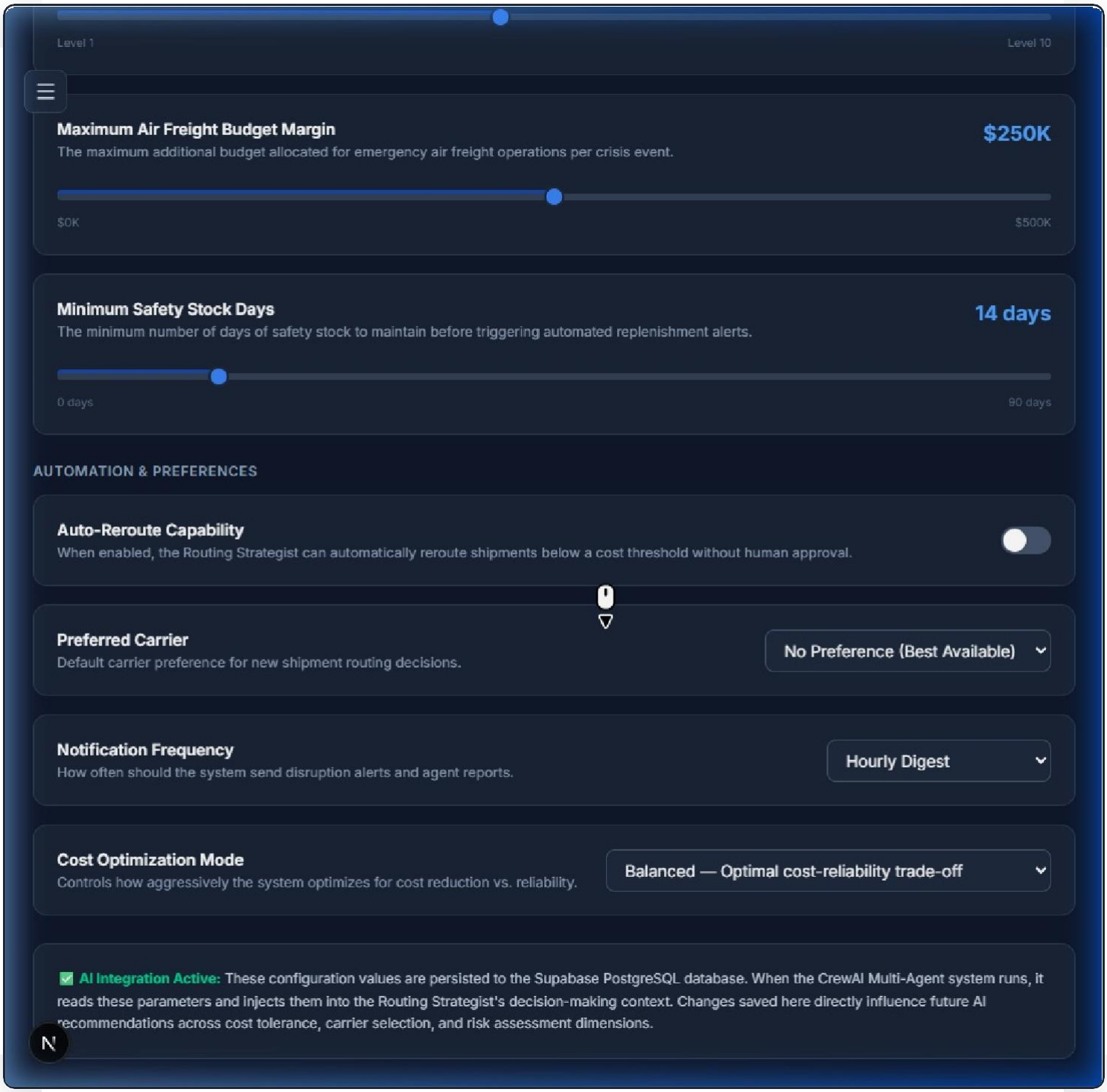


Crisis Resolution Center with collapsible scenario input panel, preset crisis buttons, and Run AI Analysis trigger

Explanation: This screenshot shows the Crisis Resolution Center page — the primary interface where users interact with the CrewAI system. Key elements include a collapsible Crisis Scenario Input Panel with crisis description textarea, affected routes input, preset crisis buttons (Red Sea Crisis, Japan Port Earthquake, Rotterdam Port Strike), and a "Run AI Analysis" button that triggers the full CrewAI pipeline via the FastAPI backend.

Below the input panel, three Agent Output Cards display AI-generated outputs: Global Risk Scout Analysis, Inventory Forecaster Impact, and Routing Strategist Plans.

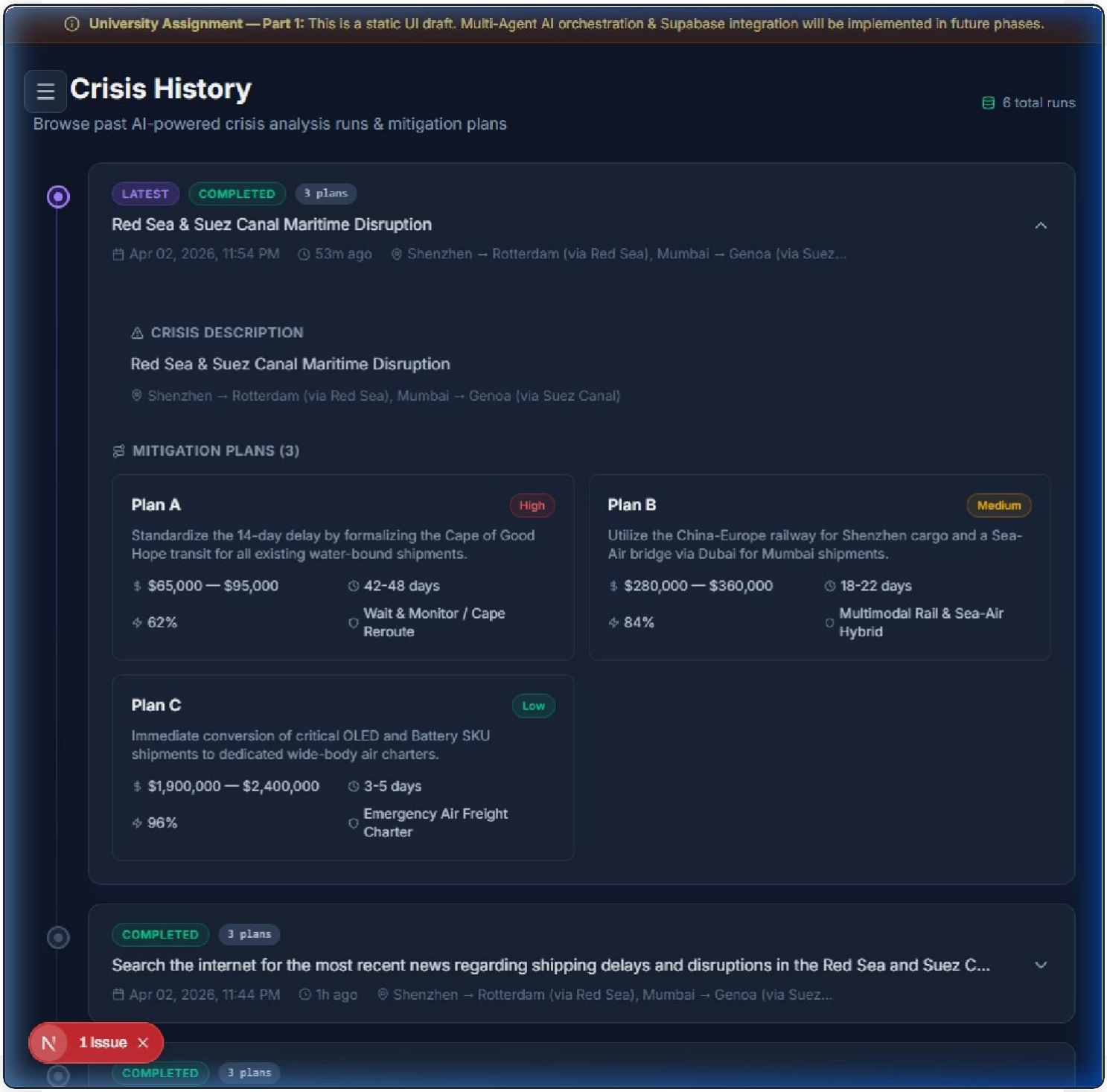
10.2 Agent Configuration Settings



Settings page showing corporate parameters that influence AI agent decisions

Explanation: This screenshot shows the Settings page where users configure corporate parameters that directly influence AI agent decisions. Settings are stored in the Supabase `user_settings` table and loaded at runtime. Key parameters include Risk Tolerance Level (slider 1-10), Maximum Air Freight Budget, Minimum Safety Stock Days, Preferred Carrier, Auto-Reroute, and Cost Optimization Mode.

10.3 Crisis History — Audit Trail



Crisis History page showing logged past crisis runs with AI outputs

Explanation: This screenshot shows the Crisis History page, providing a full audit trail of all past CrewAI analysis runs stored in the Supabase `crisis_runs` table with crisis descriptions, affected routes, intermediate AI outputs, mitigation plans, timestamps, and status.

---

11. File Structure

```
backend/
├── .env                          # API keys (GEMINI, SERPER, SUPABASE)
├── requirements.txt              # Python dependencies
└── src/
    ├── api.py                    # FastAPI REST endpoint
```

```
    └── supply_chain_crew/
        ├── __init__.py
        ├── crew.py                # CrewAI orchestration (@CrewBase)
        ├── main.py                # CLI kickoff + Supabase persistence
        ├── models.py              # Pydantic structured output models
        ├── tools.py               # Custom Supabase inventory tool
        └── config/
            ├── agents.yaml        # Agent role/goal/backstory definitions
            └── tasks.yaml         # Task descriptions + expected outputs
```

---

12. Dependencies (requirements.txt)

```
crewai[tools,google-genai]>=0.100.0
python-dotenv>=1.0.0
supabase>=2.0.0
fastapi>=0.100.0
uvicorn>=0.23.0
pydantic>=2.0.0
```

---

13. How to Run

Terminal (CLI Mode):

```
cd backend
.venv\Scripts\python -m supply_chain_crew.main
```

API Mode (for Frontend Integration):

```
cd backend
.venv\Scripts\python -m uvicorn src.api:app --reload --port 8000
```

Then from the Next.js frontend, clicking **"Run AI Analysis"** triggers the full pipeline via:

```
Next.js → /api/crew/run (proxy) → FastAPI :8000 → CrewAI Agents → Supabase → Frontend
```

---

14. Conclusion

The CrewAI integration transforms the Smart Supply Chain Optimizer from a static Next.js mockup into a **live, AI-powered crisis resolution platform**. The three-agent pipeline (Risk Scout → Forecaster → Strategist) demonstrates:

1. **Multi-Agent Orchestration**: Specialized agents with distinct roles collaborating sequentially
2. **External Tool Integration**: Web search (SerperDev) and database queries (Supabase)

3. **Structured Output**: Pydantic models enforcing type-safe JSON that maps directly to frontend TypeScript interfaces
4. **Human-in-the-Loop**: AI generates recommendations, but humans authorize execution
5. **Settings-Driven AI**: Corporate parameters dynamically influence agent decisions at runtime
6. **Full-Stack Integration**: Python CrewAI backend → FastAPI REST API → Next.js frontend → Supabase database
7. **Audit Trail**: Every crisis analysis is logged to the database for historical review